

Data Access

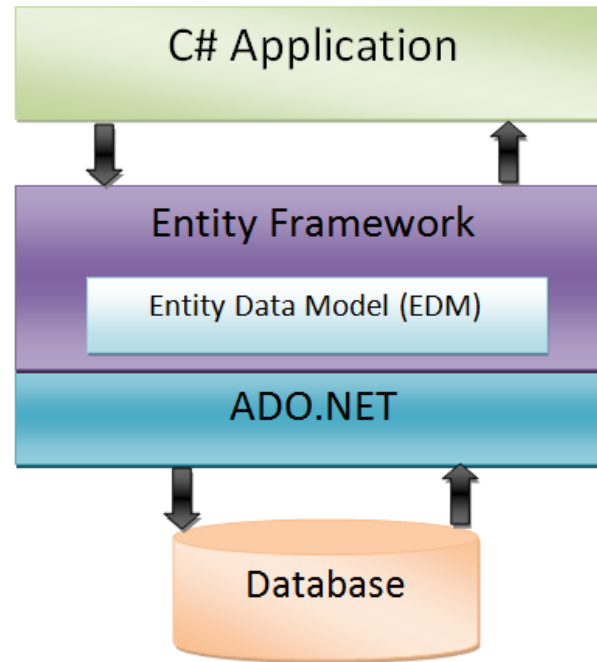
- ◆ Define and describe the Entity Framework
- ◆ Explain how to work with the Entity Framework
- ◆ Define and describe how to initialize a database with sample data
- ◆ Explain how to use LINQ queries to perform database operations

Entity Framework

- ◆ To address the data access requirements of ASP.NET MVC application, you can use an ORM framework.
- ◆ An ORM framework:
 - ◆ Simplifies the process of accessing data from applications.
 - ◆ Performs the necessary conversions between incompatible type systems in relational databases and object-oriented programming languages.
- ◆ The Entity Framework is an ORM framework that ASP.NET MVC applications can use.
- ◆ The Entity Framework is an implementation of the Entity Data Model (EDM), which is a conceptual model that describes the entities and the associations they participate in an application.
- ◆ EDM allows you to handle data access logic by programming against entities

Entity Framework

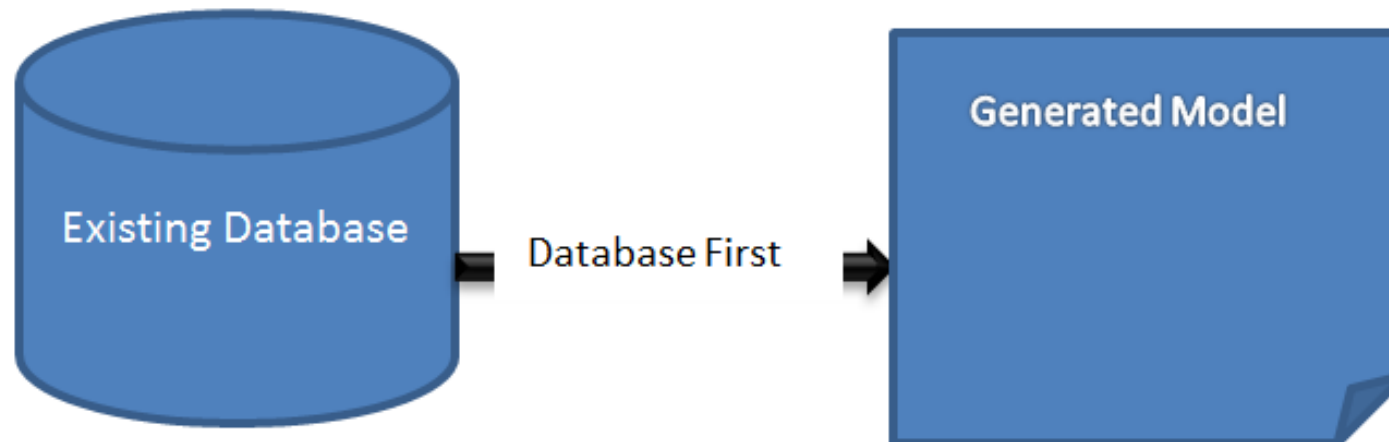
- ◆ Following figure shows the role of EDM in the Entity Framework architecture:



- ◆ Entity Framework:
 - ◆ Eliminates the need to write most of the data-access code that otherwise need to be written.
 - ◆ Uses different approaches, such as database-first and code-first to manage data related to an application.

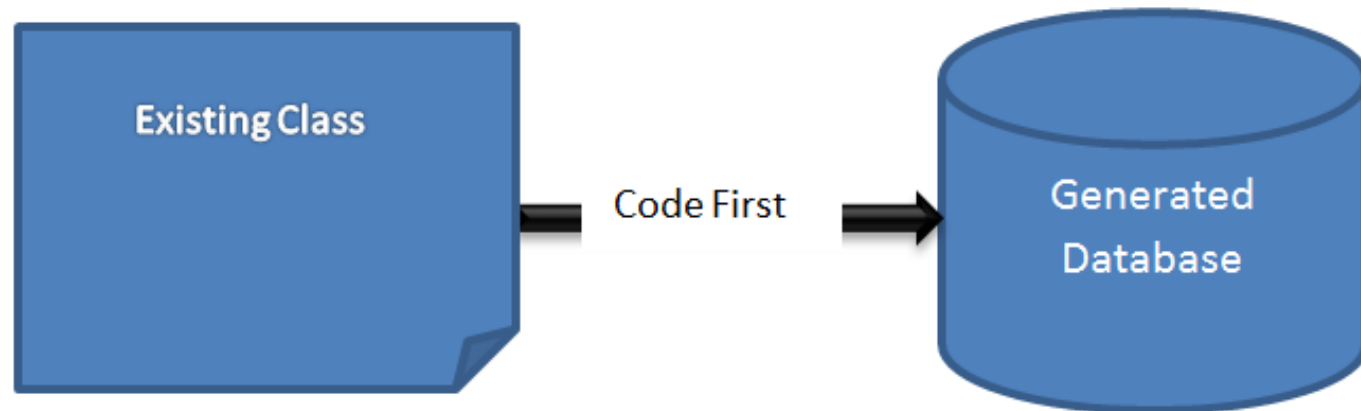
Database-first Approach

- ◆ In the database-first approach the Entity Framework creates model classes and properties corresponding to the existing database objects, such as tables and columns.
- ◆ The database-first approach is applicable in scenario where a database already exists for the application.
- ◆ Following figure shows the database-first approach:

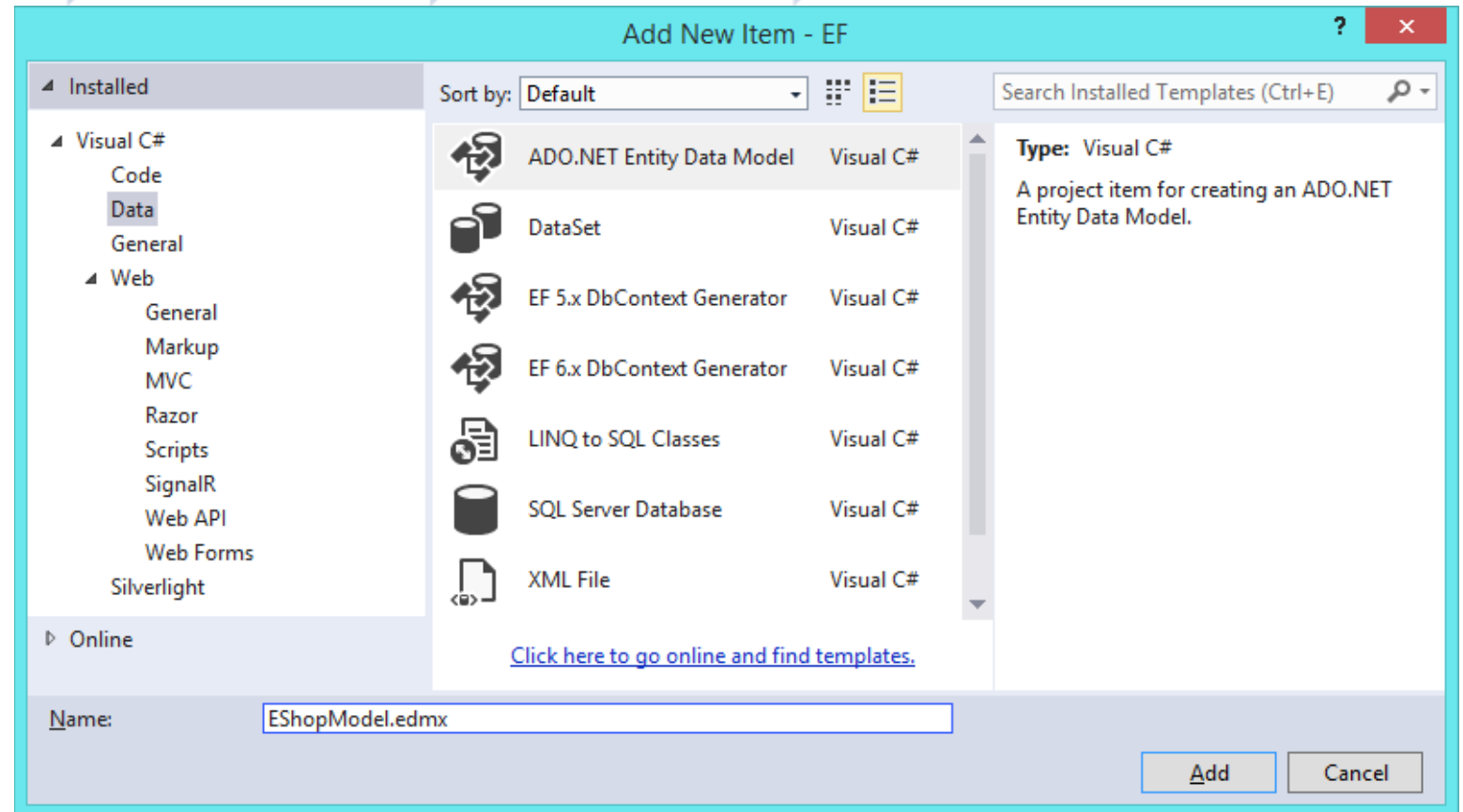
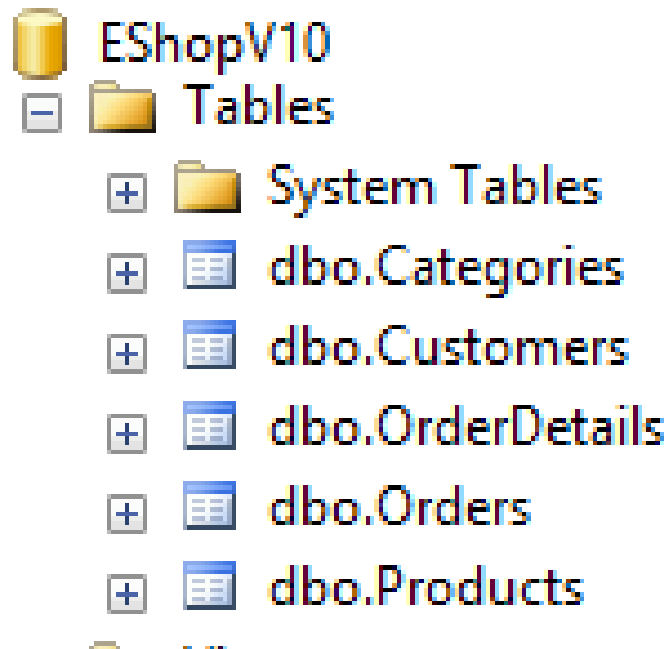
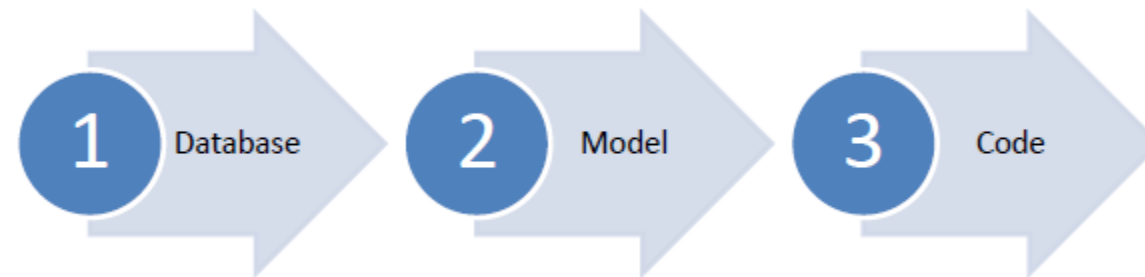


Code-first Approach

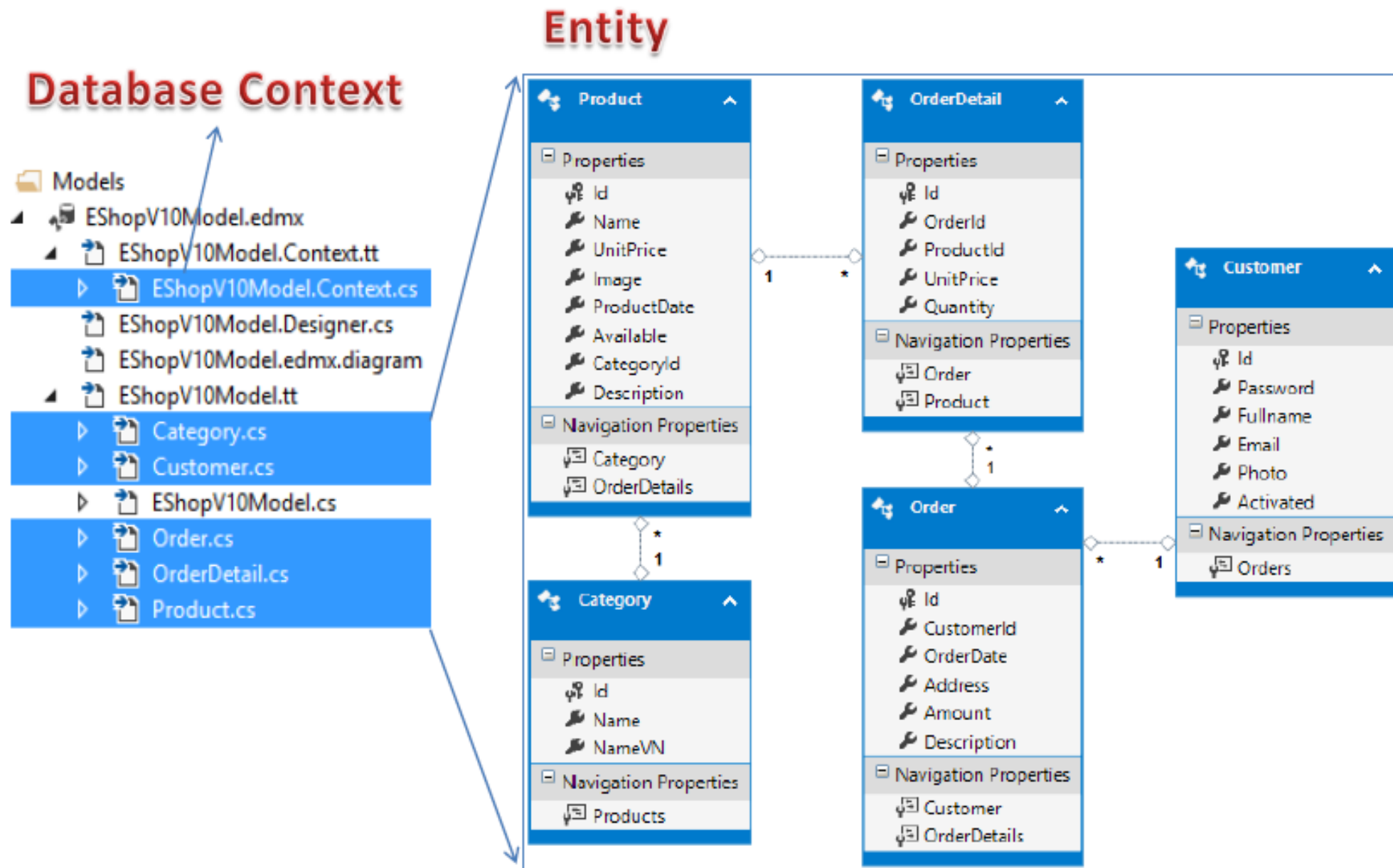
- ◆ In the code-first approach the Entity Framework creates database objects based on model classes that you create to represent application data.
- ◆ The code-first approach:
 - ◆ Is the most common approach implemented in ASP.NET MVC Framework.
 - ◆ Allows you to develop your application by coding model classes and properties and delegate the process of creating the database objects to the Entity Framework.
- ◆ Following figure shows the code-first approach:



Database-first Approach



Database model



Database context

```
public partial class EShopV10 : DbContext
{
    public EShopV10()
        : base("name=EShopV10")
    {
    }

    public DbSet<Category> Categories { get; set; }
    public DbSet<Customer> Customers { get; set; }
    public DbSet<OrderDetail> OrderDetails { get; set; }
    public DbSet<Order> Orders { get; set; }
    public DbSet<Product> Products { get; set; }
}
```



```
<connectionStrings>
    <add name="EShopV10" connectionString="..."
        providerName="System.Data.EntityClient" />
</connectionStrings>
```

Entity

- ◆ Entity name $\Leftrightarrow$ Table name
- ◆ Attribute name $\Leftrightarrow$ Column name
- ◆ Navigation properties 1-many

```
public partial class Category
{
    public Category()
    {
        this.Products = new HashSet<Product>();
    }

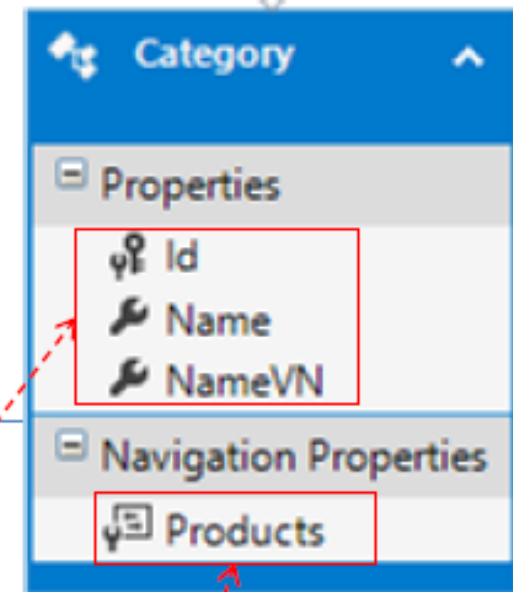
```

```
    public int Id { get; set; }
    public string Name { get; set; }
    public string NameVN { get; set; }

```

```
    public virtual ICollection<Product> Products { get; set; }
}

```

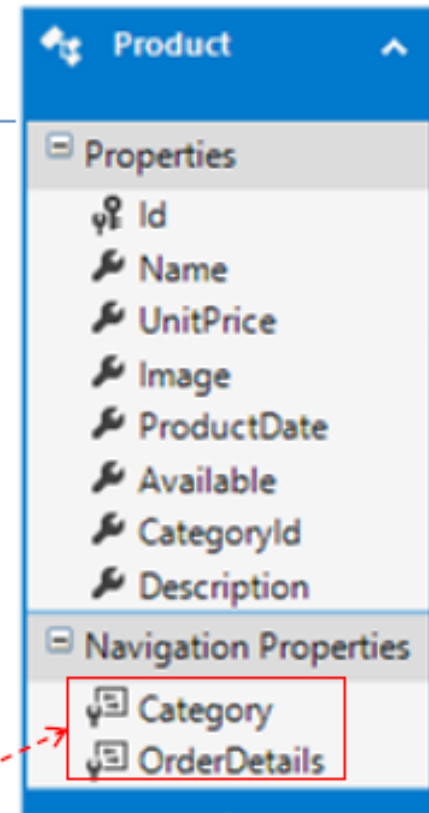


Navigation properties

```
public partial class Product
{
    public Product()
    {
        this.OrderDetails = new HashSet<OrderDetail>();
    }

    public int Id { get; set; }
    public string Name { get; set; }
    public double UnitPrice { get; set; }
    public string Image { get; set; }
    public System.DateTime ProductDate { get; set; }
    public bool Available { get; set; }
    public int CategoryId { get; set; }
    public string Description { get; set; }

    public virtual Category Category { get; set; }
    public virtual ICollection<OrderDetail> OrderDetails { get; set; }
}
```



Entity Framework API

◆ Tạo đối tượng db_context

```
var db = new EShopV10();
```

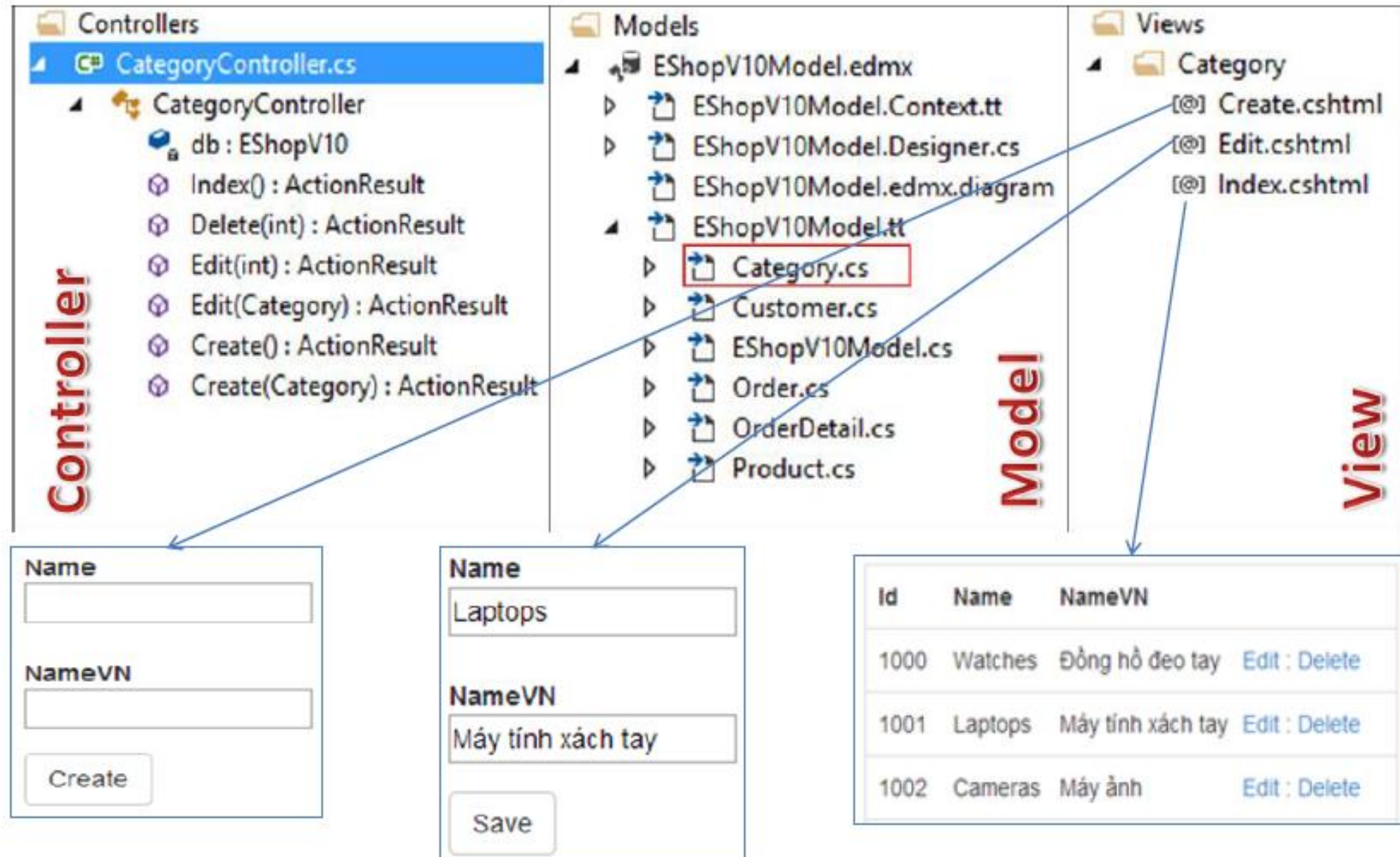
◆ Thao tác & truy vấn thực thể

Thêm mới thực thể	db.Categories. Add (category)
Cập nhật thông tin của thực thể	db. Entry (category). State = EntityState.Modified
Xóa thực thể	db.Categories. Remove (category)
Truy vấn một thực thể theo mã	var entity = db.Categories. Find (id)
Truy vấn tất cả các thực thể	var list = db. Categories

◆ Lưu sự thay đổi

```
db.SaveChanges();
```

Practice



Create template

```
// GET:/Category/Create
public ActionResult Create()
{
    return View();
}

// POST:/Category/Create
[HttpPost]
public ActionResult Create(Category model)
{
    // Thêm mới
    db.Categories.Add(model);
    db.SaveChanges();
    return RedirectToAction("Index");
}
```

Name

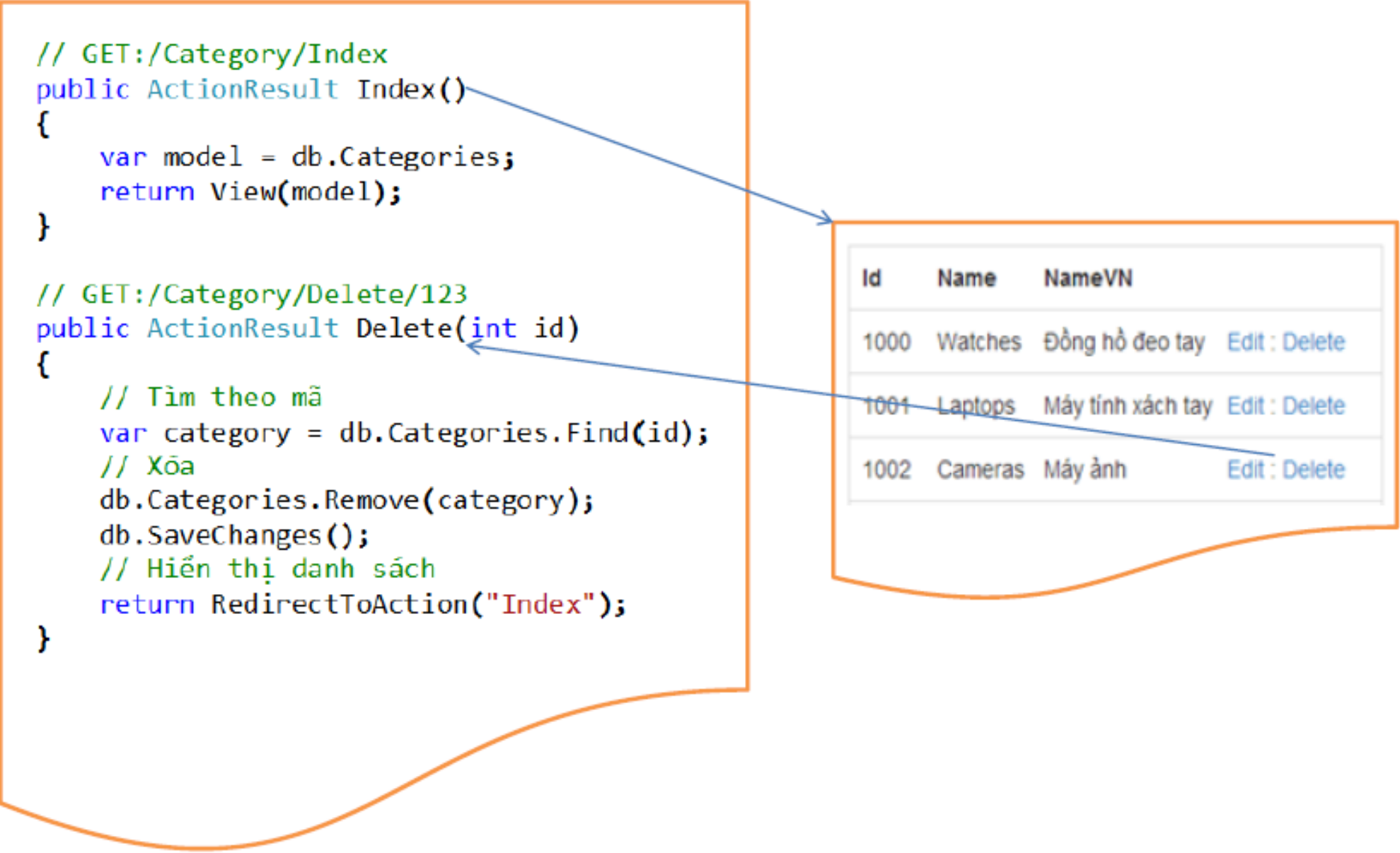
NameVN

Create

List và Delete template

```
// GET:/Category/Index
public ActionResult Index()
{
    var model = db.Categories;
    return View(model);
}

// GET:/Category/Delete/123
public ActionResult Delete(int id)
{
    // Tìm theo mã
    var category = db.Categories.Find(id);
    // Xóa
    db.Categories.Remove(category);
    db.SaveChanges();
    // Hiển thị danh sách
    return RedirectToAction("Index");
}
```



The diagram illustrates the relationship between the code and the UI. A blue arrow points from the `Index()` method to the table, indicating it provides the data. Another blue arrow points from the `Delete(int id)` method to the row with `Id` 1001, indicating it handles the deletion of that specific record. An orange bracket at the bottom groups the code and the table together.

Id	Name	NameVN	
1000	Watches	Đồng hồ đeo tay	Edit : Delete
1001	Laptops	Máy tính xách tay	Edit : Delete
1002	Cameras	Máy ảnh	Edit : Delete

Edit template

```
// GET:/Category/Edit/123
public ActionResult Edit(int id)
{
    var model = db.Categories.Find(id);
    return View(model);
}

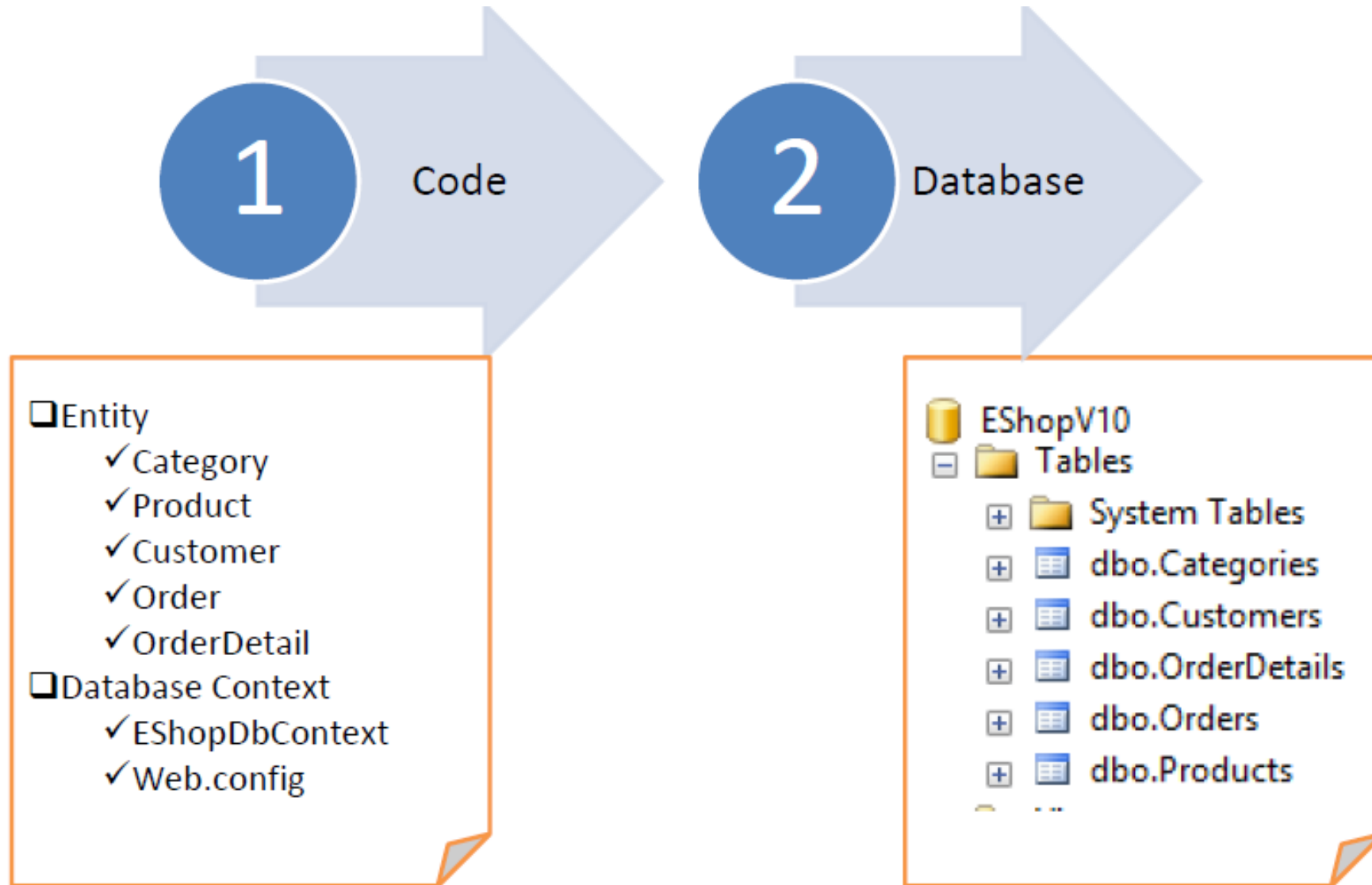
// POST:/Category/Edit
[HttpPost]
public ActionResult Edit(Category model)
{
    db.Entry(model).State = EntityState.Modified;
    db.SaveChanges();
    return View(model);
}
```

Id	Name	NameVN	
1000	Watches	Đồng hồ đeo tay	Edit : Delete
1001	Laptops	Máy tính xách tay	Edit : Delete
1002	Cameras	Máy ảnh	Edit : Delete

Name

NameVN

Code-first Approach



Entity

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public double UnitPrice { get; set; }
    public string Image { get; set; }
    public DateTime ProductDate { get; set; }
    public bool Available { get; set; }
    public int CategoryId { get; set; }
    public string Description { get; set; }

    public virtual Category Category { get; set; }
    public virtual List<OrderDetail> OrderDetails { get; set; }
}

public class Category
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string NameVN { get; set; }

    public virtual List<Product> Products { get; set; }
}

public class Order
{
    public int Id { get; set; }
    public string CustomerId { get; set; }
    public DateTime OrderDate { get; set; }
    public string Address { get; set; }
    public double Amount { get; set; }
    public string Description { get; set; }

    public virtual Customer Customer { get; set; }
    public virtual List<OrderDetail> OrderDetails { get; set; }
}

public class OrderDetail
{
    public int Id { get; set; }
    public int OrderId { get; set; }
    public int ProductId { get; set; }
    public double UnitPrice { get; set; }
    public int Quantity { get; set; }

    public virtual Order Order { get; set; }
    public virtual Product Product { get; set; }
}

public class Customer
{
    public string Id { get; set; }
    public string Name { get; set; }
    public string NameVN { get; set; }
    public string Address { get; set; }
    public string Phone { get; set; }
    public string Email { get; set; }

    public virtual List<Order> Orders { get; set; }
}
```

Database Context

- ◆ using namespace System.Data.Entity;

```
public class EShopV10 : DbContext
{
    public EShopV10() : base("name=EShopV10") { }

    public DbSet<Category> Categories { get; set; }
    public DbSet<Customer> Customers { get; set; }
    public DbSet<OrderDetail> OrderDetails { get; set; }
    public DbSet<Order> Orders { get; set; }
    public DbSet<Product> Products { get; set; }
}
```

```
<connectionStrings>
  <add name="EShopV10"
    connectionString="Server=.;Database=EShopV10;Integrated Security=True"
    providerName="System.Data.SqlClient" />
</connectionStrings>
```

```
[Table("KhoaHoc")]
public class Course
{
    [Key]
    [Column("MaKH")]
    public int Id { get; set; }
    [Column("TenKH")]
    public String Name { get; set; }
    [Column("HocPhi")]
    public double UnitPrice { get; set; }
}
```

Data initialization

```
public class MvcApplication : System.Web.HttpApplication
{
    protected void Application_Start()
    {
        Database.SetInitializer(new MusicStoreDbInitializer());
    }
}
```



```
public class MusicStoreDbInitializer
    : DropCreateDatabaseIfModelChanges<MusicStoreDbContext>
{
    protected override void Seed(MusicStoreDbContext context)
    {
        base.Seed(context);
        var genres = new List<Genre>
        {
            new Genre { Name = "Rock" },
        };
        var artists = new List<Artist>
        {
            new Artist { Name = "Aaron Copland & London Symphony Orchestra" },
        };
        var albums = new List<Album>
        {
            new Album { Title = "A Copland Celebration, Vol. I", Genre :
            };
        };
        albums.ForEach(a => context.Albums.Add(a));
    }
}
```

◆ LINQ: **L**anguage **I**Ntegrated **Q**uery

2 types of LINQ

◆ Query

```
var result = from s in students  
            where s.Marks > 9  
            orderby s.Marks descending  
            select new { s.Name, s.Marks };
```

Biểu thức truy vấn

Kiểu nội bộ tự suy

Kiểu nặc danh

Khởi tạo đối tượng

◆ Method

```
var result2 = students  
    .Where(s => s.Marks > 9)  
    .OrderByDescending(s => s.Marks)  
    .Select(s => new { s.Name, s.Marks });
```

Biểu thức lambda

Phương thức mở rộng

◆ Using LINQ to query Customers

```
[IQueryable<Customer> q = from s in db.customers select s;
```

```
string names = "";  
IEnumerable<Customer> q = from s in db.customers select s;  
foreach (var cust in q)  
{  
    names = names + " " + cust.Name;  
}  
ViewBag.Name = names;
```

```
public static void DisplayCustomerNames() {  
    using (EShopDbContext db = new EShopDbContext()) {  
        IQueryable<String> query = from c in db.Customers select c.Name;  
        Console.WriteLine("Customer Names:");  
        foreach (String custName in query){  
            Console.WriteLine(custName);  
        }  
    }  
}
```

```
public static void DisplayCustomerByName() {  
    using (EShopDbContext db = new EShopDbContext()) {  
        IQueryable<Customer> query = from c in db.Customers  
                                     where c.Name == "Alex Parker" select c;  
        Console.WriteLine("Customer Information:");  
        foreach (Customer cust in query)  
        {  
            Console.WriteLine("Customer ID: {0}, Name: {1},  
                             Address: {2}", cust.CustomerId, cust.Name, cust.Address);  
        }  
    }  
}
```

```
IQueryable<Customer> q = from s in db.customers  
    where s.City == "New Jersey"  
    orderby s.Name ascending  
    select s;
```

```
var q = from s in db.customers  
        groups by s.City;
```

```
public static void DisplayPropertiesMethodBasedQuery() {  
    using (EShopDbContext db = new EShopDbContext()) {  
        var cusNames = db.Customers.Select(c => c.Name);  
        Console.WriteLine("Customer Names and Addresses:");  
        foreach (var n in cusNames)  
        {  
            Console.WriteLine("Name: {0}", n);  
        }  
    }  
}
```



Lambda

```
// Khai báo delegate
public delegate void SimpleDelegate(int x, int y);
// Khởi tạo delegate sử dụng biểu thức Lambda
SimpleDelegate simpleDelegate = (x, y) =>
{
    Console.WriteLine(x);
    Console.WriteLine(y);
};
// Gọi delegate
simpleDelegate(2,3);
```

```
public static void DisplayPropertiesMethodBasedQuery() {  
    using (EShopDbContext db = new EShopDbContext()) {  
        var query = db.Customers.Select(c=> new {CustomerName = c.Name,  
                                                CustomerAddress = c.Address});  
        Console.WriteLine("Customer Names and Addresses:");  
        foreach (var custInfo in query)  
        {  
            Console.WriteLine("Name: {0}, Address: {1}",  
                              custInfo.CustomerName, custInfo.CustomerAddress);  
        }  
    }  
}
```

Phương thức	Mô tả	Ví dụ
.Where(e=>điều kiện)	Lọc	Students.Where(s=>s.Marks > 9)
.GroupBy(e=>biểu thức)	Nhóm	Students.GroupBy(s=>s.Clazz)
.OrderBy(e=>biểu thức) .OrderByDescending(e=>biểu thức)	Sắp xếp	Students.OrderBy(s=>s.Name)
.Select(e=>đối tượng)	Chọn	Students.Select(s=>new{s.Name, s.Marks})
.Distinct()	Giữ 1 của các đối tượng giống nhau	Numbers.Distinct()

```
public static void DisplayPropertiesMethodBasedQuery() {  
    using (EShopDbContext db = new EShopDbContext()) {  
        var query = db.Customers.Where(c => c.Name="John");  
        Console.WriteLine("Customer Names and Addresses:");  
        foreach (var custInfo in query)  
        {  
            Console.WriteLine("Name: {0}, Address: {1}",  
                custInfo.Name, custInfo.Address);  
        }  
    }  
}
```



Lambda

```
public static void DisplayPropertiesMethodBasedQuery() {  
    using (EShopDbContext db = new EShopDbContext()) {  
        var query = db.Customers.Where(c => c.Name="John")  
            .Select(c => new {CustomerName=c.Name,  
                             CustomerAddress=c.Address});  
  
        Console.WriteLine("Customer Names and Addresses:");  
        foreach (var custInfo in query)  
        {  
            Console.WriteLine("Name: {0}, Address: {1}",  
                custInfo.CustomerName, custInfo.CustomerAddress);  
        }  
    }  
}
```

```
public static void DisplayPropertiesMethodBasedQuery() {  
    using (EShopDbContext db = new EShopDbContext()) {  
        var query = db.Customers.OrderByDescending(c => c.Id)  
            .Select(c => new {CustomerName=c.Name,  
                            CustomerAddress=c.Address});  
        Console.WriteLine("Customer Names and Addresses:");  
        foreach (var custInfo in query)  
        {  
            Console.WriteLine("Name: {0}, Address: {1}",  
                custInfo.CustomerName, custInfo.CustomerAddress);  
        }  
    }  
}
```

```
public static void DisplayPropertiesMethodBasedQuery() {  
    using (EShopDbContext db = new EShopDbContext()) {  
        var groups = db.Products.GroupBy(p => p.CategoryId)  
                                .Select(g => new { CatId = g.Key, Num = g.Count()});  
        Console.WriteLine("Customer Names and Addresses:");  
        foreach (var g in groups)  
        {  
            Console.WriteLine("Category: {0}, NumOfProduct: {1}",  
                              g.CatId, g.Num);  
        }  
    }  
}
```